EdgeSched: 面向资源受限边缘设备的轻量级多模态 AI 任务调度系统

作者姓名
学院名称
大学名称
城市, 国家
email@example.com

摘要—边缘设备上部署大语言模型与图像生成模型面临显存约束与任务异构的双重挑战。8GB 显存无法同时容纳 LLM（4-bit 量化约 4.7GB + KV Cache）与图像生成模型（约 6.7GB），而现有方案要么面向数据中心（Ray、Kubernetes），要么假设充足 GPU 资源（vLLM、FlexGen），缺乏针对边缘设备的轻量级调度解决方案。

本文提出 EdgeSched，一个面向边缘设备的多任务 AI 调度系统。针对显存硬约束与用户体验软约束的非对称性，我们设计了动态资源迁移机制：一种结合" 紧急模式"（Urgent Mode）用于快速抢占，以及" 原子热切换"（Atomic Hot-Swap）用于毫秒级恢复的策略。系统支持双任务调度、背压控制、取消语义和崩溃恢复（< 15ms），提供完整的任务生命周期管理。

在 RTX 5070 Laptop（8GB 显存）上的实验表明，EdgeSched 在混合负载下实现了 100% 的成功率（基线方案因 OOM 失败），在模型恢复期间将服务中断控制在 300ms 以内，且相比原始信号量方案调度开销低于 11%。代码已开源：https://github.com/[YourRepo]

***Index Terms*—边缘计算, 资源调度, 大语言模型, 图像生成, GPU 内存管理**

I. 引言

A. 背景与动机

近年来，边缘 AI 应用快速增长。用户希望在本地设备上运行大语言模型（LLM）和图像生成模型，以保护隐私、降低延迟。典型场景包括：边与 LLM 对话边生成图像，或在本地完成文档理解与配图创作。然而，边缘设备资源严重受限——主流笔记本仅配备 8GB 显存，而 7B 参数的 LLM（Q4_K_M 量化后模型权重约 4.7GB，加上 KV Cache 约 1-2GB，总计约 6-7GB）与 SDXL Turbo 图像模型（fp16 精度约 6.7GB）无法同时驻留显存。这种资源约束与多模态需求的矛盾，迫切需要轻量级的任务调度系统。

B. 核心挑战

显存硬约束。8GB 显存无法同时容纳 LLM（模型权重约 4.7GB + KV Cache 约 1-2GB，总计约 6-7GB）和图像模型（约 6.7GB），传统 GPU 隔离方案不可行。当图像生成任务到达时，必须立即释放 LLM 占用的显存，否则触发 CUDA OOM。

任务异构性。不同任务具有截然不同的特性，如表 **??**所示：

表 I
任务特性对比

任务类型	资源需求	延迟敏感度	可排队性	运行语义
LLM 对话	GPU/CPU	高（流式）	低	常驻交互
图像生成	GPU 必需	中（端到端）	高	一次性任务

LLM 需要持续低延迟响应，而图像生成可以排队等待。这种异构性要求调度系统支持非对称的资源迁移策略。

C. 现有方案的局限

现有方案主要面向两个方向：（1）数据中心分布式调度（如 Ray、Kubernetes），启动时间在秒级到分钟级，内存占用 GB 级别，不适合资源受限的边缘设备；（2）单任务推理优化（如 vLLM、FlexGen），假设有充足的 GPU 资源，缺乏在显存不足时迁移到 CPU 的能力。

边缘设备上的多模态 AI 调度问题尚未得到充分研究。现有方案无法解决 8GB 显存设备上 LLM 与图像生成模型共存的资源竞争问题，缺乏轻量级、低开销的调度解决方案。EdgeSched 填补了这一空白，提供了专为边缘设备设计的多模态任务调度机制。

D. 我们的贡献

本文提出 EdgeSched，一个面向边缘设备的多模态 AI 调度系统，核心贡献包括：

1) **轻量级架构**。调度器启动时间毫秒级，内存占用约 27MB，适合资源受限的边缘设备。
2) **非对称动态资源迁移**。针对显存硬约束与用户体验软约束的非对称性，设计紧急模式（GPU→CPU 立即释放）与原子热替换模式（CPU→GPU 预加载），将服务中断时间从秒级压缩到毫秒级。
3) **完整的任务生命周期管理**。支持优先级队列、背压控制、取消语义和崩溃恢复（<12ms），提供可预测的调度行为。
4) **开源实现与验证**。在 RTX 5070 Laptop（8GB 显存）上验证，混合负载下成功率达 100%，服务中断时间 <300ms。

EdgeSched 填补了边缘设备上多模态 AI 调度系统的空白，为资源受限场景下的 LLM 与图像生成共存提供了实用解决方案。

II. 背景

A. 边缘设备特点

边缘设备通常具有以下特点：

- **资源受限**：主流笔记本配备 8GB 显存、16-32GB 内存，远低于数据中心服务器
- **单机部署**：不需要分布式协调，但要求系统轻量高效
- **稳定性要求高**：家庭/工业场景下，系统崩溃会严重影响用户体验

B. 多模态任务特性

在边缘设备上，典型的 AI 任务可以按三个维度分类：

模型大类：

- **LLM（Transformer）**：CPU/GPU 均可推理，但边缘侧” 可用的 CPU 兜底” 很大程度依赖量化与高效推理栈（GGUF + llama.cpp 最典型）
- **Diffusion**：图像生成（理论可 CPU，但在边缘侧通常延迟不可接受，因此工程上视为 GPU 必需）

运行语义：

- **常驻任务（Resident）**：需要长期在线、持续响应，不能长时间无输出/无响应；默认对应 LLM（交互式）
- **一次性任务（One-shot）**：提交后跑完就结束；典型是图像生成

设备亲和：

- **GPU 必需（GPU-required）**：理论上可能能 CPU 跑，但在边缘设备上 CPU 时延不可接受，因此工程上视为必须 GPU 跑
- **CPU/GPU 均可（CPU/GPU-capable）**：既能 CPU 跑也能 GPU 跑，但” 是否达到边缘可用时延” 高度依赖模型与推理栈

C. 设计目标

基于上述背景，EdgeSched 的设计目标包括：

1) **轻量级**：调度器本体轻量（启动快、内存小），真实模型内存/显存另计
2) **关键任务可用性优先**：GPU 上尽量低 TTFT；迁移到 CPU 允许退化但持续可用
3) **高稳定性**：崩溃恢复时间 < 12ms，避免任务状态丢失
4) **易用性**：简单 API，开箱即用，无需复杂配置

III. 相关工作

A. 边缘计算与 *AI* 推理

边缘计算将计算能力下沉到网络边缘，以降低延迟并保护隐私 [**?**]。随着大模型的发展，边缘智能（Edge Intelligence）成为研究热点 [**?**]。然而，现有工作主要关注云端到边缘的模型部署，缺乏对资源受限设备上多模态任务共存问题的深入探讨。

B. 分布式调度框架

Ray [**?**] 和 Kubernetes [**?**] 是主流的分布式调度框架，但它们面向数据中心设计，启动时间在秒级到分钟级，内存占用 GB 级别，不适合边缘设备的资源受限场景。EdgeSched 专注于单机边缘设备，启动时间在毫秒级，内存占用约 27MB。

C. GPU 内存管理与推理优化

vLLM [?] 提出 PagedAttention 技术解决 KV Cache 碎片化问题，FlexGen [?] 实现了 GPU-CPU Offloading 方案。然而，这些方案主要面向服务器场景，假设有充足的 GPU 资源。EdgeSched 针对 8GB 显存的边缘设备，需要更激进的资源迁移策略和任务调度机制。

D. 故障恢复机制

传统分布式系统的容错方案 [?] 依赖外部存储和协调服务，开销较大。EdgeSched 使用 mmap 实现轻量级状态持久化，无需外部依赖，恢复时间 <12ms。

E. 与现有方案对比

表 **??**展示了 EdgeSched 与现有方案的对比。

表 II
与现有方案对比

方案	目标场景	启动时间	内存占用	GPU 管理	崩溃恢复
Ray	数据中心	秒级	GB 级	分布式	依赖外部
Kubernetes	云端集群	分钟级	GB 级	容器隔离	依赖外部
vLLM	服务器	秒级	GB 级	PagedAttn	无
FlexGen	单 GPU	秒级	灵活	Offloading	无
EdgeSched	**边缘设备**	**毫秒级**	**27MB**	**动态迁移**	**mmap**

EdgeSched 填补了边缘设备上多模态 AI 调度系统的空白，提供了轻量级调度、动态资源迁移、轻量级崩溃恢复和完整的任务生命周期管理。

IV. 系统设计

A. 架构概览

图 **??**展示了 EdgeSched 的整体架构。系统采用分层设计，自上而下分为表示层、领域层、控制面、执行层和数据适配层。用户请求通过 WebSocket 接口进入系统，经领域层业务逻辑处理后，由控制面统一路由到相应的执行组件。

图 1. EdgeSched 系统架构

控制面包含三个核心组件：

1) **GlobalTaskScheduler** 作为 L1 路由层，负责任务分发，管理优先级队列、取消事件和图像任务背压控制
2) **CPPSchedulerEngine** 作为 L2 高性能执行层，封装 llama-cpp-python 推理引擎，实现 GPU/CPU 动态资源迁移
3) **StateBackplane** 基于 mmap 实现零拷贝状态持久化，支持崩溃后 12ms 内恢复任务状态

执行层包含 ResourceManager，负责协调模型生命周期和 GPU/CPU 资源分配，仲裁多模型间的资源竞争。数据适配层包含 LLM Adapter 和 Image Adapter，分别封装大语言模型（GGUF 格式）和图像生成模型（Stable Diffusion）的推理逻辑，屏蔽底层推理引擎的差异，支持多种后端无缝切换。

B. 双任务调度

EdgeSched 针对边缘设备上最常见的两类大模型应用——对话交互（LLM）与图像生成（Diffusion）——设计双任务调度机制。LLM 属于常驻交互任务（Resident），需要持续响应用户输入，优先常驻 GPU 以保障低延迟；图像生成属于一次性任务（One-shot），计算密集且在 CPU 上延迟不可接受，工程上必须依赖 GPU。

系统采用不同的调度策略处理两类任务。算法 **??**展示了核心调度逻辑。LLM 任务通过 `submit_llm_task()` 直接提交至 CPPSchedulerEngine，默认占用 GPU 资源；图像生成任务通过 `try_enqueue_image_generation()` 进入独立的有界队列。当图像任务到达时，若 LLM 占用 GPU 显存，系统立即触发动态资源迁移：保存 KV Cache 后释放显存，供图像任务加载执行。若多个图像任务连续到达，后续任务在队列中排队，队列满时触发背压机制拒绝新任务，避免资源耗尽导致系统崩溃。

这种设计确保常驻任务（LLM）的持续可用性，同时为突发任务（图像生成）提供可预测的资源分配策略。

C. 动态资源迁移

边缘设备配备独立显卡时，显存与系统内存通过 PCIe 总线分离。LLM 常驻 GPU（含 KV Cache 约 6-7GB）可保障低延迟交互，但 8GB 显存无法同时容纳 LLM 与图像生成模型（约 6-7GB）。当图像生成任务到达时，必须立即释放 LLM 占用的显存，否则触发 CUDA OOM。此时显存成为硬约束，用户体验退化为软约束——短暂的服务降级优于系统崩溃。反之，当图像任务完成、显存空闲时，将 LLM 迁回 GPU 恢复高

Algorithm 1 带背压控制的任务调度

```
function 提交任务 (task)
if task.type == LLM then
    提交到 CPPSchedulerEngine
    默认分配 GPU
else if task.type == IMAGE then
    if queue.size() < total_slots then
        任务入队
        if LLM 在 GPU 上 then
            触发卸载到 CPU()
        end if
    else
        throw 队列已满错误
    end if
end if
end function
```

性能，无需牺牲上下文连续性。这种约束条件的非对称性，决定了下行路径（GPU→CPU）采用紧急模式优先释放资源，上行路径（CPU→GPU）采用原子热切换保障服务可用。

1) 非对称迁移策略: **下行路径（GPU → CPU）：紧急模式**。当图像任务到达时，系统触发紧急迁移以立即释放显存。该模式首先通过 `save_state()` 将 KV Cache 从显存经 PCIe 复制到内存（约 200-300ms），保留对话上下文；随后立即释放 GPU 显存，消除显存碎片以避免 OOM 风险。释放完成后，系统并行启动两个操作：后台加载 CPU 版 LLM（从 DDR 到 DDR，利用 Page Cache 加速，>20GB/s），以及加载图像生成模型到 GPU（从 DDR 经 PCIe 到 GDDR）。此模式通过短暂的上下文保存延迟，换取显存安全与用户体验的兼顾。

上行路径（CPU → GPU）：原子热切换。当图像任务完成、显存空闲时，系统执行原子热切换以恢复 LLM 的高性能推理。该模式采用 Double Buffering 策略：

1) **预加载：**首先在 GPU 上全新加载 LLM 实例（从 DDR 经 PCIe 到 GDDR），此时 CPU 实例仍在服务用户请求（不中断）
2) **热迁移：**待 GPU 实例就绪后，立即暂停 CPU 实例，通过 `save_state()` 提取 KV Cache 并注入 GPU 实例（纯内存操作）
3) **切换与卸载：**最后原子替换实例指针，将新请求路由到 GPU 实例，并异步卸载 CPU 实例

此模式将服务中断时间从秒级压缩到毫秒级（仅状态注入耗时），最大化服务可用性。

2) 实现机制: 非对称迁移的实现依赖三个核心机制：

1) **并发控制：**`asyncio.Lock()` 保证实例替换的原子性，避免竞态条件
2) **状态序列化：**llama-cpp-python 提供的 `save_state()` / `load_state()` 接口实现 KV Cache 的跨设备迁移，状态数据经 PCIe 在显存与内存间传输
3) **异步清理：**通过 `asyncio.create_task()` 后台释放旧实例，避免阻塞服务

D. 背压控制

图像生成任务计算密集且显存占用大，系统采用有界队列防止资源耗尽。图像队列容量由 `queue_capacity`（默认 8）和 `max_concurrent`（默认 1）共同决定，总槽位数为两者之和（默认 9）。其中 `max_concurrent` 表示并发执行数，`queue_capacity` 表示排队等待数。

队列管理通过 `try_enqueue_image_generation()` 实现:系统使用 `asyncio.Queue(maxsize=total_slots)` 创建有界队列，若队列未满，任务入队等待执行；若队列已满，立即抛出 `RuntimeError("Image queue is full")` 拒绝请求。这种设计确保系统在高压场景下保持可预测的行为，而非无限堆积任务导致内存耗尽或响应延迟失控。

此外，系统跟踪图像任务状态（pending/running/-completed），支持任务取消和结果查询，为用户提供透明的任务生命周期管理。

E. 取消语义

系统支持用户主动取消正在执行或排队的任务，采用协作式中断策略避免资源泄漏。

LLM 流式取消。LLM 生成采用流式输出，通过 `asyncio.Event` 实现协作式取消。用户调用 `cancel_llm_request(request_id)` 后，系统设置取消标志；生成循环在每个 token 后检查标志，若标志已设

置则立即停止后续生成。此模式避免强制终止导致的显存泄漏，确保资源安全释放。

图像任务取消。排队中的图像任务直接丢弃；正在执行的任务设置取消标志，但无法强制中断（需等待完成）。系统通过 `cancel_image_job(job_id)` 接口提供取消能力，已取消任务的结果将被丢弃，不向用户推送。

局限性。系统不支持硬中断（强制杀死 GPU 任务），这是由 CUDA 显存管理机制决定的。正在运行的图像任务必须等待完成，取消仅影响结果是否推送。这种设计承认不可硬中断的边界，在资源安全与响应速度之间取得平衡。

F. 崩溃恢复

边缘设备环境不稳定，系统可能因断电或异常崩溃。StateBackplane 基于 mmap 实现零拷贝状态持久化，设计目标是在系统重启后 12ms 内恢复任务状态。

数据布局。持久化文件大小为 1MB，分为三个区域：Header（8B）存储 Magic Number 和版本号，用于验证数据完整性；Task Slots（32 个槽位，每个 48B）存储任务 ID、状态、优先级和时间戳；剩余空间预留扩展。

持久化机制。任务状态变更时，通过 `register_task()` 和 `update_task_status()` 实时写入 mmap 文件。由于 mmap 将文件直接映射到内存地址空间，读写操作等价于内存访问，无需传统 I/O 的数据拷贝，延迟仅为微秒级。

恢复流程。算法 **??**展示了恢复逻辑。系统重启时，StateBackplane 初始化读取持久化文件：验证 Magic Number 和版本号确保数据格式正确；遍历 32 个任务槽位，提取状态非零的活跃任务；将状态为 Pending 的任务重新加入调度队列；将状态为 Running 的任务标记为 Failed，避免悬挂任务。该设计确保崩溃后任务状态不丢失，提供最小可用的任务闭合语义。

Algorithm 2 崩溃恢复流程

```
function 从崩溃恢复 ()
打开 mmap 文件
if magic_number ≠ 预期值 then
    throw 数据损坏错误
end if
recovered_tasks ← []
for slot in task_slots[0:32] do
    if slot.state ≠ EMPTY then
        if slot.state == PENDING then
            重新入队到调度器
        else if slot.state == RUNNING then
            标记为 FAILED
        end if
        recovered_tasks.append(slot)
    end if
end for
return recovered_tasks
end function
```

V. 实现

A. 技术栈

EdgeSched 基于 Python 3.10 实现。核心框架：FastAPI 0.110+ 提供 WebSocket API，Uvicorn 作为 ASGI 服务器，Pydantic 2.6+ 用于数据验证。机器学习推理：llama-cpp-python 0.2+ 用于 LLM 推理（GGUF 格式），PyTorch 2.0+ 配合 Diffusers 0.25+ 用于图像生成（Stable Diffusion）。异步运行时：asyncio 实现基于协程的并发，aiohttp 作为异步 HTTP 客户端。硬件配置：NVIDIA GeForce RTX 5070 Laptop（8GB 显存），AMD Ryzen 9 8940HX，32GB DDR5，Windows 11。

B. 关键模块实现

动态资源迁移。算法 **??**展示了核心迁移逻辑。当图像任务到达时，系统保存 LLM 状态，释放 GPU 显存，并异步恢复到 CPU。热切换机制确保在 GPU 恢复期间 CPU 实例继续服务请求。

任务队列管理。图像队列使用有界容量防止资源耗尽。队列满时立即拒绝新任务并发送背压信号。

状态持久化。StateBackplane 使用 mmap 实现零拷贝 I/O。任务状态通过 struct.pack 序列化后写入内存映射文件，实现亚毫秒级恢复。

C. 部署配置

LLM 配置：

- 模型：Qwen2.5-7B-Instruct（GGUF 格式，Q4_K_M 量化）
- 模型权重：约 4.7GB
- KV Cache：约 1-2GB（上下文长度 4096）

Algorithm 3 动态 GPU-CPU 迁移

function 卸载到 CPU()
    保存 KV 缓存状态
    释放 GPU 显存
    异步加载 CPU 后端
end function

function 恢复到 GPU()
    后台预加载 GPU 实例
    等待 GPU 就绪
    原子切换：CPU → GPU
end function

- 总显存占用：约 6-7GB
- GPU 层数：默认-1（自动分配），8GB 显存环境下通常加载 25-33 层
- 首字超时：30 秒
- 最大生成 token 数：512

图像生成配置：

- 模型：SDXL Turbo 1.0（fp16 精度，约 6.7GB）
- 加载策略：按需加载，完成后卸载以释放显存
- 最大并发数：1
- 队列容量：8
- 总槽位数：9（并发 + 排队）
- 图像尺寸：512×512
- 推理步数：20

VI. 评估

A. 实验设置

我们在具有代表性的边缘设备环境中评估 EdgeSched，以验证其资源管理能力、负载下的稳定性以及故障恢复机制。

硬件环境：

- **平台**：WSL2 (Ubuntu 24.04-E) 运行于 Windows 11
- **CPU**：AMD Ryzen 9 8940HX (16 Cores, 32 Threads)
- **RAM**：32GB DDR5
- **GPU**：NVIDIA GeForce RTX 5070 Laptop (8GB VRAM)
- **存储**：NVMe SSD

软件栈：

- **Python**：3.10.19 (WSL2 环境)
- **框架**：PyTorch 2.x, asyncio
- **LLM 后端**：llama.cpp，运行 `qwen2.5-7b-instruct-q4_k_m.gguf` (4-bit 量化, 模型占用约 4.7GB)
- **图像后端**：Stable Diffusion WebUI Forge (运行 SDXL Turbo, 显存需求约 6.7GB)

评估指标：

- **TTFT (首字延迟)**：从发出请求到生成第一个 token 的时间延迟
- **服务中断时间**：资源迁移过程中 LLM 服务无法响应的时长
- **恢复时间**：从进程重启到任务状态完全恢复的时间
- **吞吐量**：并发负载下每秒完成的任务数

B. 资源迁移与共存

EdgeSched 的核心能力是支持内存密集型的 LLM（常驻任务）与图像生成（突发任务）在单张 8GB 显卡上分时复用，LLM 在抢占期间回退到系统内存（DDR）运行。

1) 迁移延迟与用户体验: 我们测量了完整迁移周期内的系统行为：(1) LLM 在 GPU 上运行，(2) 图像任务抢占（LLM 卸载到 CPU），(3) 图像任务执行，(4) LLM 回迁至 GPU。表 **??**展示了资源迁移性能。

表 III
资源迁移性能

指标	数值 (ms)	说明
GPU TTFT (理想)	26.5	无图像模型时的 GPU 基线延迟
GPU TTFT (实际)	115.2	有图像模型驻留时的延迟
CPU TTFT (稳态)	198.2	图像生成期间的稳态响应延迟
迁移卡顿	1800.8	从提交图像任务到 CPU LLM 就绪
回迁耗时	2179.6	CPU→GPU 恢复的端到端耗时
最大卡顿 (回迁)	291.6	恢复过程中用户感知的最大服务停顿
队列等待时间	10024.8	第二个图像任务的排队等待时间

分析：

- **无缝恢复**：回迁卡顿 (291.6 ms) 显著低于总回迁耗时 (2179.6 ms)。这验证了我们的**原子热切换**策略：CPU 实例持续服务，直到 GPU 实例预加载完成才进行切换。